

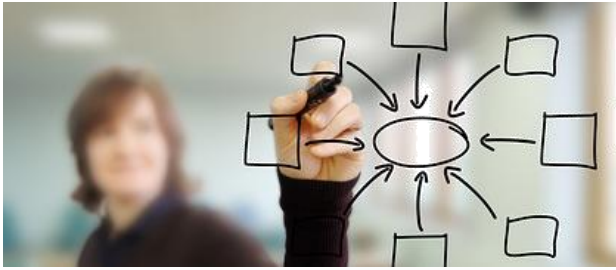


Unidad Didáctica

**Modelado y Análisis de Software,
Unidad IV**

Contenido

VÍndice de Contenido	2
Introducción	3
Desarrollo de Contenidos	4
El Modelo de proceso del software	4
Modelo de proceso.....	4
Estructura del proceso	5
Actividades del proceso.....	6
Evaluación del proceso.....	7
Modelos de proceso descriptivo.....	8
Modelo de cascada	9
Modelo de proceso incremental	10
Modelos de proceso evolutivo.....	11
Modelos Concurrentes.....	13
Modelos de proceso especializado.....	14
Modelo UML	18
El Proceso Unificado	27
Ventajas y desventajas de los modelos	30
Bibliografía y Webgrafía	30
Glosario	31



Bienvenidos a la Unidad IV de nuestro curso sobre Modelado y Análisis de Software. En esta sección, nos adentraremos en el fascinante mundo del proceso de

desarrollo de software, explorando conceptos clave, modelos y actividades estructurales que son fundamentales para comprender cómo se lleva a cabo la creación de software efectivo y eficiente.

Durante esta unidad, nos proponemos alcanzar varios objetivos fundamentales. En primer lugar, buscamos familiarizarnos con los conceptos esenciales y los diferentes modelos que guían el proceso de desarrollo de software. Además, nos enfocaremos en identificar las actividades fundamentales que se llevan a cabo en cualquier proceso de software, así como en comprender cómo se modelan estos procesos, abordando distintos enfoques de modelado de software.

Exploraremos detalladamente el Modelo de Proceso del Software, desglosando su estructura y examinando las actividades que lo componen. Posteriormente, nos sumergiremos en los diversos modelos de proceso descriptivo, como el modelo de cascada, el modelo incremental, los modelos evolutivos y los modelos concurrentes, entre otros. Además, exploraremos modelos especializados, como el desarrollo basado en componentes, el modelo de métodos formales y el desarrollo de software orientado a aspectos.

Para enriquecer nuestra comprensión práctica, dedicaremos tiempo a estudiar el Modelo UML (Lenguaje de Modelado Unificado), profundizando en sus diferentes aspectos, como casos de uso, diagramas de secuencia, diagramas de clases y

diagramas de estado. Asimismo, examinaremos el Proceso Unificado, comprendiendo su historia y las fases que lo componen.

Finalmente, evaluaremos las ventajas y desventajas de los diversos modelos de proceso, permitiéndonos reflexionar sobre cuál es el más adecuado para diferentes contextos y proyectos de desarrollo de software.

¡Prepárense para sumergirse en un viaje de aprendizaje emocionante y enriquecedor mientras exploramos los fundamentos del modelado y análisis de software!

Desarrollo de Contenidos

El Modelo de proceso del software

En el corazón del desarrollo disciplinado de software se encuentra el modelo de proceso del software, el cual define el conjunto de actividades, roles, productos y flujos de trabajo que guían la transformación de necesidades humanas y organizativas en sistemas computacionales funcionales. En palabras de Sommerville (2011), “los procesos de software son actividades estructuradas que tienen como objetivo el desarrollo o la evolución de un producto software”. Este modelo representa una visión sistemática de cómo se produce el software, alejándose de prácticas improvisadas o artesanales.

Modelo de proceso

Un modelo de proceso es una abstracción que representa las fases principales involucradas en el desarrollo de software y la forma en que estas fases interactúan. No prescribe cómo debe hacerse cada tarea en detalle, sino que ofrece un marco general que puede adaptarse a contextos específicos.

Componentes fundamentales:

Actividades: pasos o tareas del desarrollo (como análisis, diseño, codificación).

Entregables: artefactos o productos generados (como documentos, código, diagramas).

Roles: responsables de las actividades (analistas, programadores, usuarios, testers).

Flujos de trabajo: cómo y cuándo se ejecutan las actividades.

Un modelo de proceso bien definido incrementa la previsibilidad, la calidad y la capacidad de gestión del desarrollo de software.

Aplicación en Nicaragua

En entornos educativos como universidades o institutos tecnológicos nicaragüenses, contar con un modelo de proceso estructurado permite que los estudiantes desarrollen proyectos más sólidos, pasando del enfoque improvisado a uno guiado por principios metodológicos.

Estructura del proceso

La estructura del proceso se refiere a la forma en que están organizadas y relacionadas las actividades del proceso de software. La mayoría de los modelos comparten una estructura básica que puede describirse en tres niveles:

a) Marco de trabajo genérico

Propuesto por Pressman (2010), contiene las siguientes actividades esenciales:

- Comunicación con el cliente
- Planificación
- Modelado (análisis y diseño)

- Construcción (codificación y pruebas)
- Entrega y retroalimentación

b) Actividades del proceso

Estas actividades se desarrollan en ciclos iterativos o fases secuenciales, dependiendo del modelo adoptado.

c) Tareas específicas

Cada actividad se divide en tareas detalladas, por ejemplo: diseñar la base de datos, definir los casos de uso, configurar entornos de desarrollo, etc.

Reflexión local

En empresas de desarrollo emergente en Nicaragua, como startups tecnológicas en León o Managua, estructurar el proceso ha permitido mejorar la trazabilidad del software, facilitando la gestión de proyectos, incluso con recursos limitados.

Actividades del proceso

Las actividades del proceso de software no son simplemente técnicas, sino también humanas y organizativas. Estas actividades forman la columna vertebral del desarrollo:

a) Especificación de requisitos

Consiste en identificar lo que el cliente necesita que el software haga. Se realiza mediante entrevistas, cuestionarios, análisis documental y observación directa.

Ejemplo: En el diseño de un sistema para seguimiento de pacientes en clínicas rurales del norte de Nicaragua, los requerimientos surgen del personal médico,

muchas veces con conocimientos limitados de informática. Aquí la empatía y la escucha activa se vuelven habilidades técnicas clave.

b) Diseño del sistema y del software

Se especifica cómo funcionará internamente el sistema. Incluye diseño de la arquitectura, interfaces, base de datos, algoritmos, etc.

c) Implementación y prueba (codificación y verificación)

Codificación del software y pruebas sistemáticas para detectar errores y validar que cumpla con los requisitos.

d) Mantenimiento

Después de la entrega, el software se corrige, adapta y mejora continuamente. Esta etapa suele ser la más costosa y extensa.

e) Gestión del proceso

Incluye planificación de tareas, gestión de riesgos, control de calidad, seguimiento del cronograma, etc.

Evaluación del proceso

La evaluación del proceso de software permite determinar si el proceso aplicado está siendo efectivo y cómo puede mejorarse. Esta evaluación se basa en métricas, auditorías, revisiones de calidad y encuestas de satisfacción.

Modelos de evaluación internacionalmente reconocidos:

- CMMI (Capability Maturity Model Integration): clasifica la madurez de los procesos en 5 niveles.

- ISO/IEC 12207: define procesos para adquisición, suministro, desarrollo, operación y mantenimiento del software.
- SPICE (ISO/IEC 15504): evalúa procesos en términos de capacidad.

Indicadores comunes de evaluación:

- Cumplimiento de plazos y presupuestos
- Calidad del producto (errores reportados)
- Satisfacción del cliente
- Reutilización de componentes

Caso nicaragüense

En proyectos educativos financiados por el Banco Mundial para plataformas de aprendizaje digital, como los implementados por el MINED en zonas rurales, la evaluación de procesos ha sido esencial para corregir enfoques ineficaces, adaptar el software a contextos con baja conectividad y asegurar la sostenibilidad.

Modelos de proceso descriptivo

En la ingeniería de software, los modelos de proceso descriptivo representan enfoques estructurados para guiar el desarrollo de sistemas. Cada modelo surge como respuesta a necesidades específicas del contexto organizacional y tecnológico. Aunque con el tiempo han surgido metodologías ágiles y adaptativas, los modelos descriptivos clásicos siguen siendo fundamentales tanto en la teoría como en la práctica, especialmente en proyectos donde la estabilidad, la trazabilidad y la documentación son cruciales.

Según Sommerville (2011), un modelo de proceso de software es “una representación simplificada de un proceso de software, presentada desde una perspectiva particular”, lo cual implica que cada modelo no solo refleja una

estructura técnica, sino también una filosofía subyacente sobre cómo se debe construir software

Modelo de cascada

El modelo de cascada fue uno de los primeros enfoques sistemáticos para el desarrollo de software, propuesto por Winston Royce en 1970. Se caracteriza por su enfoque secuencial, en el cual cada fase debe completarse antes de que comience la siguiente.

Etapas principales:

Recolección de requerimientos

Diseño del sistema

Implementación

Verificación (pruebas)

Mantenimiento

Ventajas

Alta organización y control del proyecto.

Buena documentación.

Adecuado para proyectos donde los requerimientos están claramente definidos desde el inicio.

Desventajas

Poco flexible frente a cambios en los requerimientos.

Alto costo de corrección de errores detectados en etapas avanzadas.

No fomenta la participación activa del cliente durante el desarrollo.

Ejemplo aplicado a Nicaragua

En proyectos institucionales del gobierno, como el desarrollo de un sistema para la administración de becas del MINED, donde los requisitos están definidos desde el inicio por normativa estatal, este modelo permite una planificación ordenada y predecible.

Modelo de proceso incremental

El modelo incremental combina elementos del modelo cascada aplicados de manera repetitiva. El sistema se construye en partes o “incrementos”, cada uno de los cuales agrega funcionalidad al producto final. Es particularmente útil cuando los requisitos no están completamente definidos al inicio.

Ventajas

Posibilidad de entrega temprana de funcionalidades operativas.

Más flexible frente a cambios.

Mejor control del riesgo en cada incremento.

Desventajas

Requiere una planificación cuidadosa de los incrementos.

Puede ser más complejo gestionar versiones parciales del sistema.

Riesgo de acumulación de deuda técnica si no se integra adecuadamente cada incremento.

Ejemplo aplicado a Nicaragua

Un proyecto para desarrollar un sistema de gestión académica modular para universidades privadas en regiones como Rivas o Matagalpa podría beneficiarse de este modelo. Inicialmente se desarrolla el módulo de matrícula, luego evaluación docente, y posteriormente expedientes estudiantiles, permitiendo funcionalidad temprana y mejora continua.

Modelos de proceso evolutivo

Este enfoque se basa en la construcción de versiones sucesivas y cada vez más completas del sistema, partiendo de una versión inicial incompleta. Se adapta bien a entornos donde los requerimientos son inciertos o cambian constantemente.

Tipos comunes:

Desarrollo exploratorio: se construyen prototipos y se refina el sistema basado en la retroalimentación.

Desarrollo de prototipos desechables: se crea un prototipo temporal para comprender requerimientos y luego se descarta.

Ventajas

Muy centrado en el usuario.

Permite descubrir requisitos reales en el proceso.

Promueve una fuerte comunicación cliente-desarrollador.

Desventajas

Puede no generar documentación formal suficiente.

El desarrollo sin una estructura clara puede desembocar en caos.

Requiere mucha interacción con el usuario final.

Ejemplo aplicado a Nicaragua

En un contexto como el desarrollo de una aplicación móvil para agricultores de cooperativas en Nueva Guinea, donde no todos los requerimientos están claros desde el inicio, se puede comenzar con una funcionalidad básica (registro de cultivos), obteniendo retroalimentación directa de los productores para evolucionar

la aplicación hacia nuevas funcionalidades como predicción de cosechas o alertas meteorológicas.

Modelo en espiral

Propuesto por Barry Boehm en 1988, el modelo en espiral combina elementos del modelo cascada y del modelo evolutivo con un fuerte énfasis en la gestión del riesgo. El desarrollo se estructura en ciclos ("vueltas") que representan fases de planificación, análisis de riesgo, ingeniería, evaluación del cliente y revisión.

Fases de cada espiral:

Determinar objetivos y alternativas

Identificar y analizar riesgos

Desarrollo y validación

Planificación del siguiente ciclo

Ventajas

Excelente para proyectos grandes y complejos.

Permite un control riguroso del riesgo.

Integra retroalimentación continua del cliente.

Desventajas

Costoso y complejo de implementar.

Requiere alta especialización en análisis de riesgos.

Menos adecuado para proyectos pequeños.

Ejemplo aplicado a Nicaragua

Podría emplearse en un sistema nacional de información hospitalaria interconectada entre hospitales departamentales como el de Estelí, Chinandega y Managua. Se

iniciaría con un prototipo para consultas ambulatorias, se evalúan riesgos (seguridad de datos, interoperabilidad), se desarrollan módulos más críticos como quirófanos y emergencias, y se avanza por ciclos controlados.

Modelos Concurrentes

El modelo concurrente, también conocido como modelo concurrente de ingeniería del software (Pressman, 2010), plantea que las actividades del desarrollo de software pueden suceder de forma paralela en lugar de seguir una secuencia estricta.

Características:

Se centra en representar el estado actual de cada actividad del proceso (en espera, en ejecución, en revisión, etc.).

Soporta actividades simultáneas, como la codificación y la validación temprana de requisitos.

Utiliza una estructura de estados y transiciones, similar a una máquina de estados.

Ventajas:

Alta adaptabilidad al cambio de requerimientos.

Permite un desarrollo multidisciplinario y en paralelo.

Aumenta la eficiencia en equipos distribuidos.

Desventajas:

Requiere fuerte coordinación y comunicación.

Mayor complejidad en la gestión del proyecto.

Ejemplo aplicado en Nicaragua:

En proyectos de desarrollo de software educativo para radios comunitarias, donde equipos de diseño de contenidos, programación y prueba trabajan en paralelo, este modelo permite sincronizar entregas aunque los procesos no estén completamente lineales.

Modelos de proceso especializado

Los modelos especializados surgen para atender contextos, dominios o requisitos particulares donde los modelos genéricos (como el cascada o espiral) resultan insuficientes.

Ejemplos de procesos especializados:

- Modelos para sistemas en tiempo real
- Modelos para software embebido
- Modelos para inteligencia artificial y machine learning
- Modelos de desarrollo rápido (RAD)
- Modelos DevOps

Ventajas:

- Se ajustan con precisión a contextos específicos.
- Favorecen el cumplimiento de normas y estándares del dominio (por ejemplo, salud o banca).
- Mejoran la eficiencia operativa en industrias altamente reguladas.

Desventajas:

- Dificultad para generalizar o adaptar fuera de su contexto.
- Curva de aprendizaje alta.

Reflexión local:

En entornos como la agroindustria nicaragüense, el uso de sistemas embebidos para monitoreo de humedad del suelo (por ejemplo, en la zona de Sébaco) se beneficia de modelos especializados que integran sensores, software y controladores en una lógica de tiempo real.

Desarrollo basado en componentes

El modelo CBD propone construir sistemas a partir de componentes previamente desarrollados y bien definidos, integrándolos en arquitecturas de software modulares.

Principios clave:

Reutilización de software

Encapsulamiento de funcionalidades

Interfaces bien definidas

Ventajas:

Reducción significativa en el tiempo y costo de desarrollo.

Mayor mantenibilidad y escalabilidad.

Posibilidad de usar componentes comerciales (COTS).

Desventajas:

Dependencia de terceros y riesgo de incompatibilidades.

Complejidad en la gestión de versiones y licencias.

Posibles problemas de rendimiento y seguridad.

Aplicación nicaragüense:

En aplicaciones de gestión para cooperativas agrícolas, como sistemas de control de inventarios y ventas, el CBD permite ensamblar soluciones a partir de bibliotecas

existentes (como sistemas de autenticación, control de usuarios, reportes, etc.), reduciendo el desarrollo desde cero.

El modelo de métodos formales

Este modelo se basa en el uso de matemáticas formales para especificar, diseñar y verificar software. Utiliza lógica, teoría de conjuntos y álgebra para describir el comportamiento esperado del sistema.

Herramientas comunes: Z, B-Method, VDM, Alloy

Ventajas:

Garantía de exactitud matemática.

Reducción de errores críticos en sistemas complejos o de seguridad.

Posibilidad de verificación automatizada.

Desventajas:

Alto nivel de abstracción y dificultad técnica.

Requiere formación matemática avanzada.

Costoso en términos de tiempo y recursos.

Aplicación práctica:

Aunque poco extendido en Nicaragua, el uso de métodos formales sería relevante en sectores como la banca o la salud pública. Por ejemplo, el diseño de software para transferencias interbancarias o historia clínica electrónica nacional exige precisión y confiabilidad donde los métodos formales tendrían un gran aporte.

Desarrollo de software orientado a aspectos.

El desarrollo orientado a aspectos surge para resolver la problemática de las preocupaciones transversales (cross-cutting concerns) que afectan múltiples partes del sistema, como la seguridad, los logs, o la auditoría.

Concepto central:

Los aspectos representan funcionalidades que no pueden encapsularse fácilmente en una sola clase o módulo.

Herramientas y lenguajes: AspectJ, Spring AOP

Ventajas:

- Mejora la modularidad del código.
- Reduce la duplicación y facilita el mantenimiento.
- Separa las preocupaciones funcionales de las transversales.

Desventajas:

- Complejidad en el diseño y la depuración.
- Poca compatibilidad con herramientas tradicionales.
- No siempre es intuitivo para nuevos desarrolladores.

Ejemplo nicaragüense:

En un sistema de seguimiento de asistencia docente en universidades nicaragüenses, el registro de logs, notificaciones por correo o reglas de auditoría pueden manejarse mediante aspectos, sin contaminar la lógica de negocio central.

Modelo UML

El Lenguaje Unificado de Modelado (Unified Modeling Language, UML) es un estándar gráfico propuesto por OMG (Object Management Group) que permite representar, visualizar, especificar, construir y documentar los artefactos de un sistema de software. Su objetivo principal es facilitar la comprensión y comunicación entre analistas, diseñadores, programadores y usuarios.

Fue concebido inicialmente por Grady Booch, Ivar Jacobson y James Rumbaugh en los años noventa, y es independiente de metodologías específicas, pero compatible con enfoques como el RUP, Scrum, o el modelo de cascada.

Características principales:

Es un lenguaje visual y formalmente definido.

Soporta el diseño orientado a objetos.

Incluye diagramas estructurales y de comportamiento.

Se aplica tanto al análisis como al diseño.

Problema de ejemplo

La Universidad Nacional Autónoma de Nicaragua (UNAN) emprende con esperanza y responsabilidad la construcción de un sistema web de gestión de asistencia académica. Este sistema debe permitir a:

Docentes: registrar la asistencia de sus clases.

Estudiantes: consultar su historial personal de asistencia.

Jefes de Área: consultar reportes globales y recibir alertas cuando un estudiante acumule tres inasistencias consecutivas.

Contexto y Motivación

La UNAN, con sedes en Managua, León, Estelí, Rivas y Chontales, enfrenta retos:

Registro manual consume tiempo valioso de los docentes.

Dificultad en trazabilidad al cambiar de sección o sede.

Retrasos en detección de ausencias y activación de protocolos de acompañamiento.

Este proyecto surge del compromiso de mejorar la calidad académica y bienestar estudiantil, aliviando la carga administrativa y facilitando la toma de decisiones.

Stakeholders

<i>Rol</i>	<i>Intereses</i>
<i>Docentes</i>	Rapidez en el registro, confiabilidad de datos, simplicidad.
<i>Estudiantes</i>	Acceso a historial, transparencia, notificaciones.
<i>Jefes de Área</i>	Reportes consolidados, alertas tempranas, trazabilidad.
<i>Departamento TI</i>	Mantenibilidad, escalabilidad, seguridad, integración.
<i>Autoridades UNAN</i>	Optimización de recursos, cumplimiento de normativas.

Requerimientos Funcionales y No Funcionales

Funcionales: autenticación SSO; gestión de clases; registro y consulta de asistencia; generación de reportes y alertas; auditoría de cambios.

No Funcionales: 99% uptime; <2s de respuesta; TLS y RBAC; escalabilidad; accesibilidad web responsiva; mode offline parcial.

Restricciones, Suposiciones y Criterios de Éxito

Infraestructura variable en sedes rurales; integración con sistema legado; mensajería interna disponible; equipo TI local.

Identificadores únicos por carnet; manuales y capacitación.

Éxito: 70% reducción de tiempo de registro; alertas en <24h; adopción al 100% en primer semestre.

Casos de uso

Un diagrama de casos de uso permite modelar interacciones funcionales entre los actores (usuarios o sistemas externos) y el sistema en desarrollo. Su propósito es describir los servicios principales que proporciona el sistema, representando el qué se puede hacer, no el cómo se hace.

Elementos principales:

Actores: Entidades externas que interactúan con el sistema (personas u otros sistemas).

Casos de uso: Funciones o servicios ofrecidos por el sistema.

Relaciones:

Asociación: Conexión entre actor y caso de uso.

Include: El caso de uso base incluye la funcionalidad de otro.

Extend: El caso de uso puede extenderse condicionalmente con otro comportamiento.

Tabla Resumen: Casos de Uso

<i>Actor</i>	<i>Casos de Uso</i>
<i>Docente</i>	RegistrarAsistencia, ConsultarHistorial
<i>Estudiante</i>	ConsultarHistorial
<i>Jefe de Área</i>	ConsultarReporte, RecibirAlerta

```

@startuml UseCaseDiagram
left to right direction
skinparam packageStyle rectangle

actor Docente
actor Estudiante
actor JefeDeÁrea

rectangle "Sistema de Gestión de Asistencia" {
    usecase "RegistrarAsistencia" as UC1
    usecase "ValidarCredenciales" as UC0
    usecase "ConsultarHistorial" as UC2
    usecase "ConsultarReporte" as UC3
    usecase "RecibirAlerta" as UC4
}

Docente --> UC1
Docente --> UC2
Estudiante --> UC2
JefeDeÁrea --> UC3
JefeDeÁrea --> UC4

UC1 .> UC0 : <<include>>
UC3 .> UC4 : <<extend>>
@enduml

```

Diagramas de secuencia

Un diagrama de secuencia representa la interacción entre objetos a lo largo del tiempo, modelando cómo se comunican para cumplir un proceso.

Elementos clave:

Actor y participantes (lifelines): representan entidades activas.

Mensajes:

Síncrono: flecha continua con punta cerrada.

Asíncrono: flecha continua con punta abierta.

Retorno: flecha punteada.

Barras de activación: indican el tiempo que un objeto está activo realizando una acción.

Fragmentos (alt, loop, opt): para estructuras condicionales y repetitivas.

Tabla Resumen: Secuencia (RegistrarAsistencia)

<i>Participante</i>	<i>Responsabilidad</i>
<i>Docente</i>	Iniciar sesión, seleccionar clase, marcar asistencia
<i>InterfazWebDocente (IW)</i>	Mostrar pantallas, enviar datos al controlador, manejar errores
<i>ControladorAsistencia (CA)</i>	Validar usuario, invocar servicio de asistencia
<i>ServicioAsistencia (SA)</i>	Lógica de negocio, preparación de datos para almacenamiento
<i>BaseDeDatos (BD)</i>	Persistencia de registros de asistencia

```
@startuml SequenceDiagram
actor Docente
participant InterfazWebDocente as IW
participant ControladorAsistencia as CA
participant ServicioAsistencia as SA
database BaseDeDatos as BD

Docente -> IW : iniciarSesión()
alt credenciales válidas
    IW -> CA : validarUsuario()
    CA -> SA : obtenerClases(docenteID)
    loop por cada clase
        Docente -> IW : marcarAsistencia(clase, estado)
        IW -> CA : enviarAsistencia(datos)
        CA -> SA : registrar(datos)
        SA -> BD : insertAsistencia(datos)
    end
else credenciales inválidas
    IW -> Docente : mostrarError()
end
@enduml
```

Diagramas de clases

El diagrama de clases describe la estructura estática del sistema, mostrando las clases, sus atributos, métodos y las relaciones entre ellas. Es fundamental para el diseño orientado a objetos.

Elementos clave:

Clases: con estado y comportamiento.

Relaciones:

Asociación: conexión básica.

Agregación: relación parte-todo débil.

Composición: relación parte-todo fuerte.

Herencia: una clase especializada hereda de otra.

Visibilidad:

+ pública

- privada

protegida

Tabla Resumen: Clases

<i>Clase</i>	<i>Atributos clave</i>	<i>Métodos principales</i>
<i>Usuario (abstract)</i>	id, nombre	autenticarse()
<i>Docente</i>	hereda Usuario	registrarAsistencia()
<i>Estudiante</i>	hereda Usuario	consultarHistorial()
<i>JefeDeÁrea</i>	hereda Usuario	consultarReporte(), recibirNotificacion()
<i>Clase</i>	id, nombre, horario	—
<i>Asistencia</i>	id, fecha, estado	registrar()

```

@startuml ClassDiagram
skinparam classAttributeIconSize 0

abstract class Usuario {
    - id: String
    - nombre: String
    + autenticarse(): Boolean
}

class Docente {
    + registrarAsistencia(claseId, listaEstados): void
}
class Estudiante {
    + consultarHistorial(): List<Asistencia>
}
class JefeDeÁrea {
    + consultarReporte(): Reporte
    + recibirNotificacion(): void
}
class Clase {
    - id: String
    - nombre: String
    - horario: String
}
class Asistencia {
    - id: String
    - fecha: Date
    - estado: Enum
    + registrar(): void
}

Usuario <|-- Docente
Usuario <|-- Estudiante
Usuario <|-- JefeDeÁrea
Docente "1" -- "*" Clase : imparte >
Estudiante "*" -- "*" Clase : inscrito en >
Clase "1" -- "*" Asistencia
Asistencia o-- Clase
@enduml

```


Diagrama de Estado

El diagrama de estados muestra los diferentes estados posibles que puede tener un objeto del sistema a lo largo de su ciclo de vida, así como las transiciones entre esos estados causadas por eventos específicos.

En este caso, modelaremos el estado de un objeto Asistencia, desde que se crea hasta que se valida, notifica y almacena. También podrías modelar estados de un Estudiante (activo, suspendido, graduado, etc.) o de una Clase (programada, en curso, finalizada).

Elementos clave del diagrama

Elemento	Descripción
<i>Estado</i>	Condición o situación en la que se encuentra un objeto en un momento dado.
<i>Evento</i>	Acción que causa una transición de un estado a otro.
<i>Transición</i>	Movimiento entre estados a partir de un evento.
<i>Estado inicial</i>	Punto de partida del objeto ([*]).
<i>Estado final</i>	Estado de terminación ([*]).

Tabla resumen (Ejemplo: Objeto Asistencia)

Estado	Evento que lo activa	Estado siguiente
<i>Creado</i>	El docente inicia registro	En edición
<i>En edición</i>	Se seleccionan estudiantes y estados	Validado
<i>Validado</i>	Se verifica información y guarda	Notificado
<i>Notificado</i>	Se envía alerta al jefe de área	Finalizado
<i>Finalizado</i>	Proceso completo, ya no hay modificaciones	—

Diagrama de estados en PlantUML

```
@startuml StateDiagram
[*] --> Creado

state Creado {
    [*] --> Creado
    Creado --> EnEdicion : iniciarRegistro()
}

EnEdicion --> Validado : registrarAsistencia()
Validado --> Notificado : verificarDatos()
Notificado --> Finalizado : enviarNotificacion()
Finalizado --> [*]

@enduml
```

El Proceso Unificado

Historia

El Proceso Unificado es una metodología de desarrollo de software orientada a objetos, iterativa e incremental, propuesta inicialmente por Rational Software (ahora propiedad de IBM).

Antecedentes históricos

En la década de los 90, la comunidad de desarrollo enfrentaba desafíos importantes: crecimiento en la complejidad del software, fallos en la planificación, sobrecostos y entregas tardías.

Grady Booch, Ivar Jacobson y James Rumbaugh, tres ingenieros de software de Rational Software, propusieron integrar sus metodologías individuales en un enfoque unificado:

Booch: Metodología Booch (modelado orientado a objetos).

Jacobson: Método OOSE (Object-Oriented Software Engineering).

Rumbaugh: Método OMT (Object Modeling Technique).

Nacimiento de RUP

En 1998, Rational formalizó el Rational Unified Process (RUP) como un producto metodológico estructurado y ampliamente personalizable.

Fue diseñado para guiar a los equipos de desarrollo en la construcción de software robusto, utilizando mejores prácticas de ingeniería.

Es compatible con UML (Unified Modeling Language) como lenguaje de modelado estándar.

Fases del proceso unificado

El proceso unificado divide el ciclo de vida del software en cuatro fases principales, cada una con objetivos y entregables bien definidos. A lo largo de estas fases, se realizan iteraciones.

Fase de Inicio (Inception)

Objetivo: establecer la visión del sistema y la viabilidad del proyecto.

Identificación de los objetivos del sistema.

Estimación preliminar de costos y cronograma.

Identificación de principales actores y casos de uso.

Evaluación de riesgos iniciales.

Ejemplo:

En la UNAN, si se está proponiendo un sistema de control de asistencia estudiantil, en esta fase se define quiénes usarán el sistema (docentes, jefes de área, estudiantes), y se evalúa si es factible implementar el sistema con los recursos tecnológicos actuales.

Fase de Elaboración (Elaboration)

Objetivo: refinar los requerimientos y definir la arquitectura base del sistema.

Modelado detallado de los requisitos (diagramas UML, análisis de casos de uso).

Diseño de una arquitectura sólida (componentes, bases de datos, interfaces).

Establecimiento de un plan más detallado del proyecto.

Ejemplo:

Se definen los módulos principales del sistema: registro de asistencia, alertas, autenticación, y se diseña la base de datos con usuarios, roles, clases, etc.

Fase de Construcción (Construction)

Objetivo: construir el sistema completo o parte funcional de él.

Desarrollo iterativo e incremental del software.

Pruebas de integración, validación funcional.

Entrenamiento de usuarios claves.

Ejemplo:

El sistema comienza a desarrollarse por módulos: primero el módulo de autenticación, luego asistencia, luego alertas. Se hacen entregas funcionales parciales a los usuarios.

Fase de Transición (Transition)

Objetivo: distribuir el producto al usuario final y ajustar según realimentación.

Pruebas beta, despliegue en ambiente real.

Documentación técnica y manuales de usuario.

Corrección de errores y ajustes finales.

Ejemplo:

Se instala el sistema en los laboratorios de la UNAN, se capacita a los docentes, se prueba con clases reales, y se recopilan sugerencias para la versión definitiva.

Ventajas y desventajas de los modelos

Ventajas

<i>Ventaja</i>	<i>Explicación</i>
<i>Iterativo incremental</i>	Permite desarrollar y entregar partes funcionales del sistema a lo largo del tiempo.
<i>Arquitectura robusta</i>	En la fase de elaboración se define una arquitectura técnica bien estructurada.
<i>Control de riesgos</i>	Se identifican y mitigan los riesgos desde etapas tempranas.
<i>Adaptabilidad</i>	Puede adaptarse a diferentes tipos de proyectos y equipos.
<i>Documentación clara</i>	Se generan artefactos UML que mejoran la comprensión del sistema.

Desventajas

<i>Desventaja</i>	<i>Explicación</i>
<i>Complejidad</i>	Su implementación completa puede ser pesada y difícil para equipos pequeños.
<i>Costos iniciales altos</i>	Requiere de planificación, modelado y documentación detallada.
<i>Dependencia de herramientas</i>	Puede necesitar herramientas específicas como IBM Rational, Visual Paradigm, etc.
<i>Curva de aprendizaje</i>	Requiere conocimiento previo de UML y modelado orientado a objetos.

Bibliografía y Webgrafía

Montilva, J. C., & Barrios, J. A. (2021). Ingeniería del Software: Un enfoque basado en procesos (Edición en Español). Universidad de Los Andes.

Martel, A. (2014). Gestión práctica de proyectos con Scrum: Desarrollo de software ágil para el Scrum Master (Aprender a ser mejor gestor de proyectos) (3rd ed., Edición en Español). CreateSpace Independent Publishing Platform.

Simões, G. S., & Vazquez, C. E. (2018). Ingeniería de Requisitos: Software Orientado al Negocio (Spanish Edition). Independently published.

Pressman, R. S. (2010). Ingeniería de Software (Spanish Edition) (7th ed.). McGraw-Hill Interamericana Editores S.A. de C.V.

Agile y Scrum: Descubra el poder de la gestión de proyectos Agile, Lean Thinking, el proceso Kanban y Scrum (Spanish Edition)– 19 Diciembre 2020

Edición en Español de Robert McCarthy (Author)

McCarthy, R. (2020). Agile y Scrum: Descubra el poder de la gestión de proyectos Agile, Lean Thinking, el proceso Kanban y Scrum (Spanish Edition). Independently published.

Glosario

- **Actor:** Entidad externa que interactúa con el sistema, como un usuario, otro sistema o un dispositivo. Ejemplo: En un sistema de asistencia estudiantil, los docentes, jefes de área y estudiantes son actores.
- **Artefacto:** Cualquier producto tangible generado durante el desarrollo del software, como documentos, modelos, código fuente, pruebas, etc.
- **Arquitectura de software:** Estructura fundamental del sistema que define los componentes principales, sus relaciones y principios de diseño.
- **Caso de uso** (Use Case): Descripción de una secuencia de interacciones entre un actor y el sistema que proporciona un resultado de valor para el actor. Es central en la modelación de requisitos.
- **Ciclo de vida del software:** Conjunto completo de fases por las que atraviesa un producto software desde su concepción hasta su retiro.
- **Construcción** (Construction): Tercera fase del RUP. En ella se desarrollan y prueban los componentes del sistema, de forma incremental.

- **Diagrama de actividades:** Tipo de diagrama UML que modela el flujo de trabajo o los procesos dentro del sistema.
- **Diagrama de clases:** Diagrama estructural UML que representa las clases del sistema, sus atributos, métodos y relaciones.
- **Diagrama de estados:** Representa los distintos estados por los que puede pasar un objeto a lo largo de su vida en respuesta a eventos.
- **Documento de visión:** Artefacto que describe la visión general del sistema, los objetivos del proyecto y los requisitos principales de alto nivel.
- **Elaboración** (Elaboration): Segunda fase del RUP. Se detallan los requisitos, se diseña la arquitectura y se realiza un plan más preciso del proyecto.
- **Entregable:** Resultado tangible que se entrega al cliente o a la organización, como parte del cumplimiento de una fase del proyecto.
- **Evaluación de riesgos:** Proceso de identificación y análisis de los factores que pueden poner en peligro el éxito del proyecto.
- **Fase:** Cada una de las divisiones del ciclo de vida del RUP: Inicio, Elaboración, Construcción y Transición.
- **Inicio** (Inception): Primera fase del RUP. Se define el alcance del sistema, su viabilidad técnica y económica, y los principales actores y casos de uso.
- **Iteración:** Unidad de desarrollo dentro de una fase. Consiste en una mini-proyecto completo con análisis, diseño, implementación y prueba de una parte funcional del sistema.
- **Modelado:** Proceso de representar conceptos, estructuras y comportamientos del sistema mediante diagramas UML u otras herramientas.
- **Modelo de negocio:** Representación de los procesos de una organización y cómo estos interactúan con el sistema de software.
- **Objetivo del proyecto:** Resultado deseado o propósito que se busca alcanzar con el desarrollo del sistema.
- **Objeto:** Instancia de una clase que contiene atributos y métodos, y puede interactuar con otros objetos.

- **Proceso Unificado** (RUP): Metodología de desarrollo de software orientada a objetos, iterativa e incremental, que divide el ciclo de vida en cuatro fases principales.
- **Prueba** (Testing): Proceso de verificación del comportamiento del software frente a los requisitos. Se realiza en cada iteración y en la fase de transición.
- **Requisito funcional**: Acción o función específica que el sistema debe cumplir. Ejemplo: permitir al docente registrar la asistencia.
- **Requisito no funcional**: Calidad o condición que el sistema debe satisfacer, como rendimiento, seguridad, disponibilidad, etc.
- **Roles**: Conjunto de responsabilidades asignadas a una persona o entidad dentro del equipo de desarrollo.
- **Transición** (Transition): Fase final del RUP. Se entrega el producto al usuario, se entrena a los usuarios y se ajusta el sistema según la retroalimentación.
- **Trazabilidad**: Capacidad de relacionar requisitos, casos de uso, código, pruebas y otros artefactos entre sí, para asegurar consistencia y seguimiento.
- **UML (Unified Modeling Language)**: Lenguaje de modelado estándar para representar el diseño y análisis del sistema mediante diagramas estructurados y de comportamiento.
- **Validación**: Proceso que garantiza que el software cumple con las necesidades y expectativas del usuario final.
- **Verificación**: Proceso que asegura que el producto fue construido correctamente, conforme a los requisitos y especificaciones.
- **Visión**: Documento o declaración que presenta de forma resumida qué se quiere lograr con el sistema y por qué es necesario.